SALESFORCE EXPERIENCE CLOUD

Customisation tiers for Salesforce websites

Where design work ends, where developer effort begins, and where customisation starts to put Salesforce upgrades at risk.

Five tiers
Configure, Style, Layout, Component, Override. Each tier adds capability and cost.

Developer gate
Between Style and Layout. Beyond it, changes are code: built, tested and deployed.

Upgrade gate
Between Component and Override. Beyond it, each Salesforce release can break the site.

Summary

A Salesforce website can be customised at five levels. Designers prototype and test designs as Lightning web components in the Lightning Web Runtime (LWR), and those prototypes are what they hand over to developers to build in Salesforce. The first two, configuring in Experience Builder and styling with CSS, are designer work: no code is deployed and Salesforce maintains everything underneath. Most brand and content requirements can be met here.

Custom layouts and custom components require developers, but are built on supported, documented APIs and survive upgrades. The fifth tier, overriding or working around standard components, is where upgrade risk sits. We recommend treating the developer gate as a funding decision and the upgrade gate as an architecture approval.

The tiers at a glance

	TIER	WHAT CHANGES	WHO	UPGRADE RISK
1	**Configure**	Pages, standard components, navigation, theme settings, audiences in Experience Builder	Admin, designer	None
2	**Style**	Brand applied through styling hooks and site-level CSS on markup you own	Designer with CSS	None
DEVELOPER GATE	Code, source control, testing and deployment from here down			
3	**Layout**	Custom theme, page & component layouts built using standard base components	Developer, designer	Low
4	**Component**	New components with their own markup, data, logic and Builder properties	Developer	Low, owned by us
UPGRADE GATE	Depends on Salesforce internals that can change without notice			
5	**Override**	Changing how standard components look or behave by reaching into their internal markup	Developer	**High**

Figure 1. Tiers in order of cost. Effort and ongoing maintenance rise with each tier.

Tier 1 · Configure

Everything set through Experience Builder without writing code: choosing an LWR template, adding pages, placing standard components, setting theme colours, fonts and spacing, building navigation menus and targeting content to audiences.

Constraints. Only the options Salesforce exposes. Page structure is limited to the template's theme layouts and the standard section and column components. A component can only be adjusted through the properties shown in its Builder panel.

Tier 2 · Style

The brand applied with CSS. Standard components are styled through **styling hooks**: published CSS custom properties (the `--dxp` site hooks and `--slds-c` component hooks) that Salesforce commits to supporting. Site-level CSS, custom fonts and imagery are added as static resources. A designer who writes CSS can own this tier.

Constraints. Standard components encapsulate their markup, so CSS can change only what their hooks expose. If a hook does not exist for a property, that property cannot be changed safely at this tier. Free-form CSS is safe only on markup we write ourselves.

THE DEVELOPER GATE

A request crosses into developer effort when it needs any of the following:

- A page structure or region arrangement the template does not offer
- Data the standard components cannot display
- New behaviour: interaction, logic, validation or integration
- A new setting for authors in the Builder panel
- A visual change no styling hook covers, on a component we would need to rebuild

Past this point, work is written as code, kept in source control, tested, and deployed between environments. Designers still own the visual specification.

Tier 3 · Layout

Custom theme layouts (header, footer and page frame) and custom page layouts, built as Lightning web components that declare regions authors can fill in Builder. This is the usual first step past the gate, and it is mostly markup and CSS, so designers and developers can work closely on it.

Constraints. Requires deployment through metadata rather than editing in Builder. Each layout must be responsive and accessible, which becomes our responsibility rather than Salesforce's. Built on public APIs, so upgrade risk is low.

Tier 4 · Component

New Lightning web components with their own markup, styling, data access (including Apex), integrations and Builder properties. This covers anything the standard component set cannot do.

Constraints. Code runs under Salesforce's browser security model, and third-party scripts and domains must be explicitly allowed. Performance, accessibility and caching become our responsibility. Components are upgrade-safe because they use supported APIs, but we maintain them for as long as they exist.

THE UPGRADE GATE

Salesforce ships three major releases a year. A customisation crosses into upgrade risk when it depends on something Salesforce has not committed to keeping stable. Warning signs:

- A CSS selector targets a class name or element we did not write
- `!important` is used to beat a standard component's styles
- JavaScript modifies a standard component after it renders
- The approach was found by inspecting markup in the browser, not in documentation
- A standard component has been copied and edited instead of rebuilt

Tier 5 · Override

Changing standard components by reaching past their supported interface: CSS aimed at internal markup, scripts that alter the page after load, or reliance on undocumented behaviour.

Constraints. These changes often work at first and fail later. A release can rename an internal class or restructure a component, and the override then breaks silently on the live site. Salesforce support does not cover these issues. Each override needs regression testing against every release preview.

Recommendations

1. **Design to Tiers 1 and 2 by default.** Designers work from the list of available styling hooks so designs can be delivered without code.
2. **Treat the developer gate as a funding decision.** Each Tier 3 or 4 item is estimated and prioritised as development work, with an owner for ongoing maintenance.
3. **Treat the upgrade gate as an architecture approval.** Tier 5 needs explicit sign-off and a documented reason. Prefer rebuilding as a Tier 4 component.
4. **Keep a register of anything past the upgrade gate.** Retest each entry against Salesforce's release previews before every release.